

Ejercicio 1 – Transformaciones

Dado el siguiente método funcional que recibe tres enteros positivos:

```
public static Map<String, Long> funcional(Integer a, Integer b, Integer c) {  
    return Stream.iterate(a+b, e -> e<c, e -> e+a*b)  
        .filter(e -> e%2==0).map(e -> "[" + e%10 + "]")  
        .collect(Collectors.groupingBy(s -> s, Collectors.counting()));  
}
```

SE PIDE:

- La transformación a un método iterativo.
- La transformación a un método recursivo final.
- Calcule el orden de complejidad del algoritmo recursivo del apartado b. Defina previamente de forma recursiva el $T(n)$ para los casos mejor y peor, suponiendo el siguiente tamaño de problema: $n = (c-e)/(a*b)$. Suponga que el orden de complejidad de la concatenación de cadenas y de operar en el map es constante.

Tiempo estimado: 45 min.

Solución

a)

```
public static Map<String, Long> iterativo(Integer a, Integer b, Integer c) {
    Map<String, Long> m = new HashMap<>();
    Integer e = a+b;
    while(e < c) {
        if(e % 2 == 0) {
            String s = "[" + e%10 + "]";
            if(m.containsKey(s)) {
                m.put(s, m.get(s) + 1);
            }
            else {
                m.put(s, 1L);
            }
        }
        e = e + a*b;
    }
    return m;
}
```

b)

```
public static Map<String, Long> rec_final(Integer a, Integer b, Integer c) {
    return rec_final(a+b,a,b,c, new HashMap<>());
}

private static Map<String, Long> rec_final
(Integer e, Integer a, Integer b, Integer c, Map<String, Long> m) {
    if(e<c) {
        if(e % 2 == 0) {
            String s = "[" + e%10 + "]";
            if(m.containsKey(s)) {
                m.put(s, m.get(s) + 1);
            }
            else {
                m.put(s, 1L);
            }
        }
        m = rec_final(e + a*b,a,b,c,m);
    }
    return m;
}
```

c)

Sea $n = (c-e)/(a*b)$ el tamaño del problema

Para la llamada recursiva tendríamos un tamaño:

$$n' = (c-(e+a*b))/(a*b) = (c-e-a*b)/(a*b) = n-1$$

Definición de $T(n)$ de forma recursiva:

Caso mejor (no entrar en el `if(e%2==0)`):

$$T(n) = T(n-1) + k$$

Caso peor (entrar en el `if(e%2==0)`):

$T(n) = T(n-1) + k + f(n)$ (siendo $f(n)$ la complejidad de operar en el map que debería ser constante)

El orden de complejidad en ambos casos sería:

$$\text{Caso A: } T(n) = aT(n-b) + n^d \log^p n$$

$$[a=1, b=1, d=0, p=0]$$

$$\Theta(T(n)) = n^{d+1} \log^p n = n^{0+1} \log^0 n = n$$